

Flipkart Orders Dataset

Practical Questions — Sessions 1 to 10

No hints. No answers. Questions only.

Dataset: flipkart_orders.csv | 515 rows × 21 columns

order_id | product_id | product_name | category | price | discount_pct | discounted_price | quantity | revenue |
customer_id | customer_name | age | city | state | payment_method | order_status | order_date | delivery_date |
delivery_days | rating | review

■ Series & DataFrame

- Q1.** Create a Pandas Series from the 'revenue' column of the Flipkart dataset and display its first 10 values.
- Q2.** Create a Pandas Series from the 'category' column. Display the unique values and total count of each category.
- Q3.** Create a DataFrame manually with 5 sample Flipkart orders containing columns: product_name, category, price, quantity, revenue.
- Q4.** Load flipkart_orders.csv and display the shape, column names, index, and data types of the DataFrame.
- Q5.** Extract the 'product_name' and 'revenue' columns from the DataFrame and confirm the result is a DataFrame, not a Series.

■ Reading & Writing Data

- Q6.** Load flipkart_orders.csv using pd.read_csv() and display the first 8 rows using head().
- Q7.** Display the last 6 rows of the Flipkart dataset using tail().
- Q8.** Use df.info() to inspect the dataset. List all columns that have missing values.
- Q9.** Use df.describe(include='all') and report the mean price, maximum revenue, and most frequent payment method.
- Q10.** Export the Flipkart dataset to a new CSV file named 'flipkart_cleaned.csv' without the index column.

■ Selecting & Filtering Data (loc & iloc)

- Q11.** Select only the columns 'order_id', 'product_name', 'category', and 'revenue' from the DataFrame.
- Q12.** Using iloc, display rows 50 to 60 and only the first 5 columns.

Q13. Using loc, display all rows where order_status is 'Delivered'.

Q14. Using Boolean conditions, filter all orders where revenue is greater than 10,000.

Q15. Filter all Electronics orders placed by customers from 'Mumbai'.

Q16. Using loc, display all orders where discount_pct is 50 and order_status is 'Delivered'.

Q17. Filter orders where rating is greater than 4.0 and category is 'Electronics'. How many such orders exist?

Q18. Using query(), find all orders where age is greater than 40 and payment_method is 'UPI'.

■ Sorting & Reindexing

Q19. Sort the Flipkart dataset by 'price' in descending order and display the top 10 most expensive products.

Q20. Sort orders by 'order_date' in ascending order and display the 5 earliest orders.

Q21. Sort the dataset by 'category' ascending and then by 'revenue' descending within each category.

Q22. After sorting by revenue, reset the index and confirm the old index is dropped.

Q23. Sort the dataset by 'rating' descending (NaN last) and display the top 10 highest-rated orders.

Q24. Reindex the DataFrame using a custom range index starting from 1001 and display the first 5 rows.

■ Handling Missing Values

Q25. Display the total count and percentage of missing values for every column in the Flipkart dataset.

Q26. Drop all rows where 'customer_name' is missing. How many rows remain?

Q27. Drop only columns where more than 40% of values are missing.

Q28. Fill missing 'age' values with the median age of the entire dataset.

Q29. Fill missing 'city' values with the string 'Unknown'.

Q30. Fill missing 'rating' values with the mean rating grouped by 'category'.

Q31. Use forward fill to fill missing 'delivery_date' values and check if any nulls remain.

Q32. Fill missing 'review' values with 'No Review'. Then confirm no nulls exist in that column.

■ Handling Duplicates & Outliers

- Q33.** Check how many duplicate rows exist in the Flipkart dataset.
 - Q34.** Drop all duplicate rows and confirm the shape of the dataset before and after.
 - Q35.** Check for duplicates only based on 'order_id'. Drop them and display the count removed.
 - Q36.** Using the IQR method, detect outliers in the 'revenue' column. Display the lower and upper bounds.
 - Q37.** Using the IQR method, find how many rows in the 'price' column are outliers.
 - Q38.** Apply the Z-score method (threshold = 3) to detect outliers in the 'revenue' column.
 - Q39.** Compare the number of outliers in 'price' vs 'revenue' using the IQR method.
 - Q40.** Cap outliers in the 'revenue' column at the 5th and 95th percentile and verify using describe().
-

■ Column Operations & Data Transformations

- Q41.** Verify the 'revenue' column is correctly calculated as `discounted_price × quantity`. Add a boolean column 'revenue_correct' to confirm.
 - Q42.** Add a new column 'final_price' = `discounted_price × quantity` (same as revenue) and display the first 5 rows.
 - Q43.** Using `apply` and `lambda`, create a new column 'price_category' — 'Budget' if `price < 1000`, 'Mid-Range' if `1000–10000`, 'Premium' if above 10000.
 - Q44.** Using `apply`, create a column 'delivery_speed' — 'Fast' if `delivery_days ≤ 3`, 'Standard' if 4–7, 'Slow' if `> 7`.
 - Q45.** Using `applymap` (or `map`), convert all string columns to uppercase. Apply this only on 'category' and 'order_status'.
 - Q46.** Add a column 'discount_amount' = `price - discounted_price` and display summary statistics for it.
 - Q47.** Add a column 'is_high_value' which is `True` if `revenue > 5000` and `order_status` is 'Delivered'.
-

■ String Operations in Pandas

- Q48.** Convert all values in the 'category' column to lowercase using `str.lower()`. Check if any inconsistencies existed before.
- Q49.** Normalize the 'customer_name' column — convert to title case using `str.title()`.
- Q50.** Check how many product names contain the word 'Pro' using `str.contains()`.
- Q51.** Using `str.replace()`, replace all occurrences of 'Delivered' with 'Completed' in the 'order_status' column.

Q52. Split the 'order_id' column on the character 'D' and extract only the numeric part into a new column 'order_number'.

Q53. Find all orders where the review contains the word 'recommended' (case-insensitive).

Q54. Count how many customer names start with the letter 'R' using `str.startswith()`.

■ Date-Time Handling

Q55. Convert 'order_date' and 'delivery_date' columns to datetime format using `pd.to_datetime()`.

Q56. Extract year, month, and day from 'order_date' into three separate columns.

Q57. Find the month with the highest number of orders.

Q58. Calculate the number of days between 'order_date' and 'delivery_date' and store it in a column 'calc_delivery_days'. Compare with 'delivery_days'.

Q59. Filter all orders placed in the year 2023.

Q60. Filter all orders placed between 1st January 2023 and 31st March 2023.

Q61. Find which day of the week has the most orders using `dt.day_name()`.

Q62. Calculate the average delivery days per category for delivered orders only.

■ Merging & Joining DataFrames

Q63. Split the Flipkart dataset into two DataFrames: `df_customer` (customer_id, customer_name, age, city, state) and `df_order` (order_id, customer_id, product_name, category, revenue, order_status). Merge them back on `customer_id` using an inner join.

Q64. Perform a left join between `df_order` and `df_customer` on `customer_id`. How many rows are in the result?

Q65. Create a small DataFrame with city-level data (city, tier: 'Tier 1'/'Tier 2'/'Tier 3'). Merge it with the Flipkart dataset on 'city' and analyze revenue by city tier.

Q66. Split the dataset into two halves by order date — orders before 2023 and orders from 2023 onward. Concatenate them and confirm the original shape is restored.

Q67. Create a product summary DataFrame (product_name, avg_rating, total_orders) and merge it with the main dataset on 'product_name'. Display the top 5 products by avg_rating.

Q68. Merge `df_customer` and `df_order` on 'customer_id' using an outer join. Identify rows where customer data is missing after the merge.